Chapter 15 — Modern Workflow & Testing

15.1 CI/CD for AI Systems

Conceptual Deep Dive

An AI change can be code, **prompt**, **model version**, **retrieval config**, **index**, or **data**. CI must treat all as versioned artifacts. Pipeline stages: lint/type-check → unit tests (deterministic parts: parsers, tools, chunkers) → **offline evals** on a fixed dataset → security scans (secrets, dependencies, image) → build image → deploy to staging → smoke + shadow traffic → canary → full rollout with automatic rollback on metric regression. Keep fast checks (seconds) on every commit and expensive evals (minutes, dollars) on PRs touching prompts/models or nightly; cache LLM calls in tests (record/replay with VCR-style fixtures) to cut cost and flakiness.

Cross-Connection Scenarios

Eval datasets (10), gateway/canary routing (13.6), observability for rollback signals (14.1).

Production Anti-Patterns

- Prompts edited in a vendor UI with no version control or review.
- Running full evals on every push (slow and expensive) or never running them.
- Tests that call live APIs and fail on rate limits.

Minimal Working Code

```
# .github/workflows/ai-ci.yml
name: ai-ci
on:
  pull_request:
    paths: ["prompts/**", "src/**", "evals/**"]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.11", cache: pip }
      - run: pip install -r requirements.txt pytest ruff mypy pip-audit
      - run: ruff check . && mypy src
      - run: pip-audit -r requirements.txt
      - run: pytest tests/unit -q
      - name: Offline eval gate
        env: { LLM_API_KEY: "${{ secrets.LLM_API_KEY }}" }
        run: python evals/run.py --dataset evals/golden.jsonl --out report.json && python evals/g
ate.py report.json
      - uses: actions/upload-artifact@v4
        with: { name: eval-report, path: report.json }
```

15.2 Automated Evaluation Gates and Regression Testing

Conceptual Deep Dive

A **gate** converts eval results into pass/fail with explicit thresholds: absolute (faithfulness ≥ 0.90), relative to baseline (no metric drops > 2 points, using confidence intervals or paired bootstrap), per-slice (no slice regresses), and budget (p95 latency, cost/request). **Model-update regression testing**: when a provider deprecates a model or you swap one, re-run the golden set, diff outputs, and review the cases that flipped from pass to fail. Maintain a **regression suite** that grows with every incident: each production failure becomes a permanent test case. Control nondeterminism: temperature 0, multiple samples per case with majority/mean, and statistical tests instead of exact equality.

Cross-Connection Scenarios

Judge calibration (10.3), A/B tests after the gate passes (10.5), incident postmortems (16).

Production Anti-Patterns

- Gates with arbitrary thresholds never revisited.
- Comparing means on 30 samples and calling a 1% change a win.
- Retraining the baseline whenever a gate fails.

Minimal Working Code

```
# evals/gate.py
import json
import random
import sys

def paired_bootstrap(base, new, n=2000, seed=0):
    rng = random.Random(seed)
    diffs = [b - a for a, b in zip(base, new)]
    means = []
    for _ in range(n):
        sample = [rng.choice(diffs) for _ in diffs]
        means.append(sum(sample) / len(sample))
    means.sort()
    return means[int(.025 * n)], means[int(.975 * n)]

def gate(report: dict, baseline: dict, abs_min=0.85, max_drop=0.02) -> list[str]:
    fails = []
    for metric, scores in report["metrics"].items():
        mean = sum(scores) / len(scores)
        if mean < abs_min:
            fails.append(f"{metric}: {mean:.3f} < {abs_min}")
        lo, hi = paired_bootstrap(baseline["metrics"][metric], scores)
        if hi < -max_drop:
            fails.append(f"{metric}: significant regression (CI {lo:.3f}..{hi:.3f})")
    return fails

if __name__ == "__main__":
    rep = json.load(open(sys.argv[1]))
    base = json.load(open("evals/baseline.json"))
    problems = gate(rep, base)
    print("\n".join(problems) or "gate passed")
    sys.exit(1 if problems else 0)
```

15.3 GitOps and Infrastructure as Code

Conceptual Deep Dive

GitOps: Git is the source of truth for desired state; a controller (Argo CD, Flux) reconciles the cluster to the repo, giving audit trails, review via PRs, and trivial rollback (git revert). Keep separate repos or directories for app code, config/prompts, and infra. Use Helm/Kustomize overlays for env differences, Terraform/Pulumi for cloud resources, sealed secrets or External Secrets for credentials, and progressive delivery (Argo Rollouts) with metric-based analysis. Feature flags decouple deploy from release and serve as kill switches.

Cross-Connection Scenarios

Kubernetes (13.4), kill switch (14.5), canary analysis from observability metrics (14.1).

Production Anti-Patterns

- `kubectl apply` from laptops (configuration drift).
- Secrets committed in plain text.
- One giant environment shared by dev, staging, and prod.

Minimal Working Code

```
# argocd Application
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata: { name: llm-service, namespace: argocd }
spec:
  project: default
  source:
    repoURL: https://github.com/example/llm-platform
    targetRevision: main
    path: deploy/overlays/prod
  destination: { server: https://kubernetes.default.svc, namespace: llm }
  syncPolicy:
    automated: { prune: true, selfHeal: true }
```

15.4 Reproducibility Pipelines and Experiment Tracking

Conceptual Deep Dive

Reproducible = same code + data + config + environment + seeds → same (or statistically equivalent) results. Track with **MLflow**, **Weights & Biases**, or **DVC** (data versioning in Git-like fashion, remote storage), plus lockfiles (uv.lock, poetry.lock), container digests, dataset hashes, and model revision SHAs. Orchestrate pipelines with Prefect, Dagster, Airflow, or Kubeflow; make each step idempotent and cache by input hash. Register models/prompts in a registry with stages (staging → production) and lineage back to data and evals. Record the full "run manifest" so any production behaviour can be traced to artifact versions.

Cross-Connection Scenarios

Fine-tune lineage (12.6), tracing with prompt versions (14.1), compliance documentation (14.5).

Production Anti-Patterns

- Notebook-only experiments with no recorded parameters.
- Unpinned dataset snapshots ("latest" export).
- Non-deterministic data splits.

Minimal Working Code

```
import hashlib
import json
import platform
import subprocess
import sys
import time

def file_hash(path: str) -> str:
    return hashlib.sha256(open(path, "rb").read()).hexdigest()[:16]

def run_manifest(config: dict, dataset_path: str, model_rev: str) -> dict:
```

```
    try:
        sha = subprocess.check_output(["git", "rev-parse", "HEAD"], text=Tr
ue).strip()
    except Exception:
        sha = "unknown"
    return {"time": time.time(), "git_sha": sha, "python": sys.version,
            "platform": platform.platform(), "config": config,
            "dataset_hash": file_hash(dataset_path), "model_revision": model_rev}

# MLflow:  import mlflow; mlflow.log_params(cfg); mlflow.log_metric("faithfulness", 0.91)
# DVC:     dvc add data/golden.jsonl && git add data/golden.jsonl.dvc
```

15.5 Deployment Strategies and Rollback

Conceptual Deep Dive

Strategies: **blue/green** (instant switch, double capacity), **canary** (gradual percentage with automated analysis), **shadow** (mirror traffic, discard output), **A/B** (measure user outcomes). For prompts and models, version-pin in config and roll out via flags with per-user bucketing. Rollback triggers: error rate, guardrail block rate, repair rate, p95 latency, cost/request, negative-feedback rate, and judge-score drift on sampled traffic. Schedule model-deprecation reviews and keep a tested fallback model.

Cross-Connection Scenarios

Online metrics (10.5), gateway routing (13.6), guardrails (14.4).

Production Anti-Patterns

- Rolling out a model swap to 100% on Friday evening.
- No automated rollback criteria.
- Fallback model never exercised, so it fails when needed.

Minimal Working Code

```
import hashlib

def bucket(user_id: str, salt: str = "exp1") -> float:
    h = hashlib.sha256(f"{salt}:{user_id}".encode()).hexdigest()
    return int(h[:8], 16) / 0xFFFFFFFF

def pick_version(user_id: str, canary_pct: float = 0.05) -> str:
    return "v2" if bucket(user_id) < canary_pct else "v1"

def should_rollback(canary: dict, baseline: dict) -> bool:
    return (canary["error_rate"] > baseline["error_rate"] * 1.5
            or canary["p95_ms"] > baseline["p95_ms"] * 1.3
            or canary["thumbs_down"] > baseline["thumbs_down"] * 1.5)

print(pick_version("user-42"))
```

15.6 Prompt and Config Management as Code

Conceptual Deep Dive

Store prompts as files (Jinja or plain templates) in Git with a schema for metadata (name, owner, version, model, variables, eval suite). Review them like code: diffs, owners, required eval passes. Separate **prompt text** from **runtime config** (model id, temperature, retrieval k) and load config from a registry or environment so rollouts and rollbacks do not need deploys. Validate templates in CI: every variable is supplied, no unescaped user input in system roles, token count under a budget, and snapshot tests on rendered output. Use content hashes as versions and log them with every request. Edge cases: template injection (user text containing template syntax), whitespace changes altering behaviour, locale-specific formatting, and hidden prompt differences

between environments.

Cross-Connection Scenarios

Prompt optimisation (6.6), traces tagged by version (14.1), eval gates (15.2), deployment flags (15.5).

Production Anti-Patterns

- Editing prompts in a dashboard with no review or history.
- Formatting user text with `str.format`, allowing injection of braces.
- Different prompts in staging and production with no diff tooling.

Minimal Working Code

```
import hashlib
import re
from string import Template

class PromptTemplate:
    def __init__(self, name: str, text: str, variables: set[str], max_chars: int = 20_000):
        found = set(re.findall(r"\$\{?(\w+)\}?", text))
        if found != variables:
            raise ValueError(f"{name}: variables mismatch {found ^ variables}")
        self.name, self.text, self.vars, self.max = name, text, variables, max_chars

    @property
    def version(self) -> str:
        return hashlib.sha256(self.text.encode()).hexdigest()[:8]

    def render(self, **kw) -> str:
        if set(kw) != self.vars:
            raise ValueError("missing/extra variables")
        out = Template(self.text).substitute(**kw)       # '$' in user text is not re-interpreted
        if len(out) > self.max:
            raise ValueError("rendered prompt exceeds budget")
        return out

p = PromptTemplate("support", "You are a support bot for $company. Question: $question&quo
t;, {"company", "question"})
print(p.version, p.render(company="Acme", question="{system} $HOME?"))
```

```
import json

def snapshot_test(template: PromptTemplate, case: dict, path: str, update: bool = False):
    rendered = template.render(**case)
    try:
        expected = json.load(open(path))["rendered"]
    except FileNotFoundError:
        expected = None
    if update or expected is None:
        json.dump({"rendered": rendered}, open(path, "w"))
        return "snapshot written"
    assert rendered == expected, "prompt output changed; review the diff and update snapshot"
    return "ok"
```

15.7 Cost Governance, FinOps and Supply-Chain Hygiene in CI/CD

Conceptual Deep Dive

Treat cost as a tested property. In CI, compute **estimated cost per request** from prompt tokens, expected output tokens and prices; fail when a change raises it beyond a budget. In production, tag spend by team, feature, model and customer; set budgets

and alerts; track cost per successful task. Levers: smaller models for easy traffic, prompt caching, batching, trimming context, semantic caching, and output length limits. **Supply chain**: pin and hash dependencies, scan with pip-audit/Dependabot, generate an SBOM, sign container images (cosign), verify model weight hashes, and restrict CI secrets to protected branches. Edge cases: forked-PR workflows exposing secrets, eval runs that call paid APIs from untrusted code, and runaway CI loops.

Cross-Connection Scenarios

Token economics (2.4), gateway budgets (13.6), artifact verification (14.5), eval gates (15.2).

Production Anti-Patterns

- No cost attribution, so nobody owns the bill.
- Running paid evals on pull requests from forks with repository secrets.
- Unpinned base images and latest model aliases.

Minimal Working Code

```
def cost_gate(old: dict, new: dict, max_increase=0.15) -> list[str]
:
    def cost(d):
        return d["tin"] / 1e6 * d["p_in"] + d["tout"] / 1e6 * d["p_out
"]
    c0, c1 = cost(old), cost(new)
    if c1 > c0 * (1 + max_increase):
        return [f"cost per request up {c1 / c0 - 1:.0%} (> {max_increase:.0%})"]
    return []

base = {"tin": 1500, "tout": 300, "p_in": 0.5, "p_out": 2.0}
print(cost_gate(base, {**base, "tin": 2600}))
```

```
from collections import defaultdict

class SpendLedger:
    def __init__(self, budgets: dict[str, float]):
        self.budgets, self.spent = budgets, defaultdict(float)

    def record(self, team: str, usd: float) -> str:
        self.spent[team] += usd
        b = self.budgets.get(team, float("inf"))
        if self.spent[team] > b:
            return "block"
        return "warn" if self.spent[team] > 0.8 * b else "ok"

led = SpendLedger({"search": 100.0})
print(led.record("search", 85.0), led.record("search", 20.0))
```